Update 5: Upstream UCX Pull Request Review, External Validation, and Middleware Hardening

Post-Submission Follow-Up to Update 4

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

April 12, 2026

Contents

1 Recap **3**

2 Pull Request Review with Real Diffs **3**
2.1 PR #11304 — Calculated rndv threshold for `ucp_tag_send_nbr()` . . . 3
2.2 PR #11305 — Adaptive TX CQ moderation . . . 4
2.3 PR #11306 — Eager inline sends for host memory when CUDA MDs are present . . 4
2.4 PR #11307 — TCP as fallback wireup transport for RC/DC aliases . . . 4
2.5 PR #11308 — Estimated bandwidth query on `ucp_ep_query` . . . 5
2.6 PR #11309 — Symmetric traffic class propagation . . . 5

3 External Validation: libmesh-rdma and the r/HPC Thread **5**

4 Contrasting Production Designs **6**
4.1 Apple MLX PR #3094 (JACCL distributed backend) . . . 6
4.2 Pure Storage TurboQuant + FlashBlade . . . 6
4.3 NVIDIA HPC-X . . . 6

5 CI Status and Infrastructure Flakiness **7**

6 Security Audit Findings **7**
6.1 UCX PRs — all clean . . . 7
6.2 Middleware — four high-severity findings . . . 7

7 Middleware Hardening: Adapting libmesh-rdma **8**
7.1 `rdma_gid_discovery.py` . . . 8
7.2 `rdma_bootstrap.py` . . . 8
7.3 `rdma_qp_state_machine.py` . . . 8
7.4 Integration with `roce_transport.py` . . . 9
7.5 Testing . . . 9

8 Next Steps and Open Questions **9**

1 Recap

Update 4 introduced libscuffedrdma's dual-QP architecture, the PMP bang-bang controller, the WFA classifier, and the landscape of upstream UCX issues that motivated six pull requests (#11304–#11309). This update collects what happened after those PRs were submitted:

- The actual code diffs landed on each branch and their current commit hashes.
- External validation that arrived in the same month, most importantly the `autoscriptlabs/libmesh-rdma` project and its accompanying r/HPC thread.
- Two contrasting designs from the production LLM systems space (Apple MLX PR #3094 and Pure Storage TurboQuant + FlashBlade).
- A security audit of all six PRs and of the libscuffedrdma middleware, triggered by a memory-safety bug the author found in libmesh-rdma's `mesh_mpi_pt2pt.c`.
- A set of hardening commits applied to libscuffedrdma's Python layer, adapting three modules from libmesh-rdma while remediating four high-severity middleware findings.

Update 4 stays as-is; everything below is additive.

2 Pull Request Review with Real Diffs

Each PR is summarised below with the exact fix landed on the branch. All hashes come from the `ndg8743/ucx` fork at the time of writing.

2.1 PR #11304 — Calculated rndv threshold for `ucp_tag_send_nbr()`

Fixes #4430. Head commit `694b20db8`. Default value flipped to `auto` so both send paths use the transport-aware threshold:

```
- {"RNDV_SEND_NBR_THRESH", "256k",
+ {"RNDV_SEND_NBR_THRESH", "auto",
   "Threshold for switching from eager to rendezvous protocol in "
   "ucp_tag_send_nbr().\n"
+ "\"auto\" means to use the same dynamically calculated threshold "
+ "as for regular sends.\n"
```

With the "auto" sentinel in place, the rendezvous threshold derivation falls through to `rndv_thresh` (the calculated value) rather than the hardcoded 256 KB:

```
- rndv_local_thresh = context->config.ext.rndv_send_nbr_thresh;
+ if (context->config.ext.rndv_send_nbr_thresh == UCS_MEMUNITS_AUTO) {
+ rndv_local_thresh = rndv_thresh;
+ } else {
+ rndv_local_thresh = context->config.ext.rndv_send_nbr_thresh;
+ }
```

This is a pure behavior-flip: no new code paths, no new allocations, and no change to any on-wire protocol. Explicit values (`UCX_RNDV_SEND_NBR_THRESH=256k`) still work for users who want the old behavior.

2.2 PR #11305 — Adaptive TX CQ moderation

Fixes #1307. Head commit `e1e419982`. Three lines in one inline function:

```
- return (txqp->unsignaled >= iface->config.tx_moderation) ? flag : 0;
+ return (txqp->unsignaled >= iface->config.tx_moderation) ||
+ (iface->tx.cq_available <= (signed)iface->config.tx_moderation)
+ ? flag : 0;
```

Without the second disjunct, fan-out workloads where each RC endpoint sends fewer than `tx_moderation` messages never produce a signaled send. The TX CQ fills up with unsignaled completions, credits are never reclaimed, and the buffer pool deadlocks. Adding the iface-level `cq_available` check forces the next send on any endpoint to be signaled once CQ credits drop to the moderation threshold.

2.3 PR #11306 — Eager inline sends for host memory when CUDA MDs are present

Fixes #4275. Head commit `39939fb4e`. Adds a memory-type cache lookup to `ucp_proto_is_inline`:

```
+ /* Look up the buffer in the memory type cache. If not found, it
+ * is host memory; otherwise it is inline-eligible only if the
+ * cache entry says host. */
+ status = ucs_memtype_cache_lookup(buffer, length, &mem_info);
+ return (status == UCS_ERR_NO_ELEM) ||
+ ((status == UCS_OK) && (mem_info.type == UCS_MEMORY_TYPE_HOST));
```

The previous code was a blanket disable: once the memtype cache became non-empty (which happens as soon as any CUDA buffer is registered), all sends lost the inline-eligible fast path, including plain host buffers. The fix restores the fast path for host memory while still excluding GPU memory.

2.4 PR #11307 — TCP as fallback wireup transport for RC/DC aliases

Fixes #4794. Head commit `1554563a5` (test fix) + parent `a2ccbcb1f` (production change). The production change is five lines in the transport alias table:

```
- { "rc_v", { "rc_verbs", UCP_TL_AUX("ud_verbs"), NULL } },
- { "rc_x", { "rc_mlx5", UCP_TL_AUX("ud_mlx5"), NULL } },
+ { "rc_v", { "rc_verbs", UCP_TL_AUX("ud_verbs"), UCP_TL_AUX("tcp"), NULL } },
+ { "rc_x", { "rc_mlx5", UCP_TL_AUX("ud_mlx5"), UCP_TL_AUX("tcp"), NULL } },
  { "rc", { "rc_mlx5", UCP_TL_AUX("ud_mlx5"), "rc_verbs",
- UCP_TL_AUX("ud_verbs"), NULL } },
- { "dc", { "dc_mlx5", UCP_TL_AUX("ud_mlx5"), NULL } },
- { "dc_x", { "dc_mlx5", UCP_TL_AUX("ud_mlx5"), NULL } },
+ UCP_TL_AUX("ud_verbs"), UCP_TL_AUX("tcp"), NULL } },
+ { "dc", { "dc_mlx5", UCP_TL_AUX("ud_mlx5"), UCP_TL_AUX("tcp"), NULL } },
+ { "dc_x", { "dc_mlx5", UCP_TL_AUX("ud_mlx5"), UCP_TL_AUX("tcp"), NULL } },
```

The `UCP_TL_AUX(...)` macro marks a transport as wireup-eligible only; it is not used for data traffic. UD remains the preferred auxiliary (lower latency). TCP is only selected when UD is unavailable, which happens precisely in the SoftRoCE / direct-connect scenario that the r/HPC post described (see Section 3).

The test-side fix (commit `1554563a5`) moves the TCP-skip decision in `test_ucp_address_v2.pack_iface_attrs` from a macro-level condition into the test body, avoiding a segfault on variants where `sender()` is not yet initialised when the skip check fires.

2.5 PR #11308 — Estimated bandwidth query on `ucp_ep_query`

Fixes #6254. Head commit `ed3d69e91`. Adds `UCP_EP_ATTR_FIELD_ESTIMATED_BW` to the public `ucp_ep_attr_t`, letting the caller specify a local/remote memory type pair and receive the summed lane bandwidth. For applications doing data-placement decisions (libscuffedrdma's own WFA classifier is one), this removes the need to duplicate UCX's internal lane-scoring code in user space. The implementation is a read-only pass over `ucp_ep_config(ep)->key.rma_bw_lanes[]` that is O(lanes) and cannot fail.

2.6 PR #11309 — Symmetric traffic class propagation

Fixes #10325. Head commit `8962d27d7`. Pure bit-field isolation fix in `uct_ib_address_unpack`: the traffic-class flag bit (bit 7) was corrupting the RoCE version bits (5–6) because the unpack code right-shifted without first masking. The fix adds an `~UCT_IB_ADDRESS_FLAG_TRAFFIC_CLASS` mask before the shift, then uses the traffic class from the packed address when the local interface has none configured. After this fix, both endpoints end up with the same QoS class regardless of which one configured it.

3 External Validation: libmesh-rdma and the r/HPC Thread

On 2026-03-16, `autoscriptlabs/libmesh-rdma` appeared on GitHub: an MIT-licensed C library (75 KB compiled) implementing RDMA networking and a 55-function MPI subset for consumer GPU clusters *without a managed switch*. The author (user Ok-Pomegranate1314) also posted to r/HPC describing exactly the problem PR #11307 addresses [2]:

> *Spent today fighting UCX's UD bootstrap on a direct-connect ConnectX-7 ring (4x DGX Spark, no switch). You already know how this goes: `ibv_create_ah()` needs ARP, ARP needs L2 resolution, L2 resolution needs a subnet that both endpoints share or a switch that routes between them. Without the switch, UCX dies in `initial_address_exchange` and takes MPICH with it.*

The crucial measurement that validates the RC-over-TCP-bootstrap architecture:

> *RC QPs don't need any of this. `ibv_modify_qp()` to RTR takes the destination GID directly. No AH object. No ARP. No subnet requirement beyond what the GID encodes. The firmware transitions the QP just fine. 77 GB/s. 11.6 µs RTT. The transport layer works perfectly on direct-connect RoCE. It's only the connection management that's broken.*

The comment thread is equally illuminating because two independent users tried to configure around the bug at the UCX layer:

- `plan-bean` suggested `UCX_TLS=^ud,ud:aux` to skip UD entirely. The OP rejected this because on RoCE, UCX's connection-manager path can still hit RDMACM, which in turn uses the same ARP-based address resolution. (This is true only when the application explicitly uses `UCP_EP_PARAM_FIELD_SOCK_ADDR`; the default `UCT`-auxiliary wireup path that PR #11307 extends does not.)
- `MissionDependent4401` proposed a three-variable configuration: `UCX_TLS=rc,tcp`, `UCX_RC_VERBS_TLS=rc`, `UCX_TCP_TLS=tcp`. This *is* the environment-variable form of what PR #11307 makes the default behavior. It works, but requires the user to know that TCP can serve as a wireup auxiliary, and to set three variables correctly.

Read carefully, the thread is a real-time record of a contributor abandoning UCX because the community consensus was that UCX could not be made to work on direct-connect RoCE. PR #11307 is the minimal in-tree fix that collapses those three environment variables into a stock UCX build, and the 77 GB/s measurement from libmesh-rdma demonstrates the underlying hardware path is already production-quality once the bootstrap is sorted out. The PR is therefore a contribution to a publicly-documented pain point with an existence proof that it is worth fixing.

A footnote on RDMACM. UCX supports two distinct wireup paths: its native UCT-auxiliary wireup (which PR #11307 extends by adding TCP as a valid auxiliary) and RDMACM-based wireup (a separate code path used when the application supplies `UCP_EP_PARAM_FIELD_SOCK_ADDR` during `ucp_ep_create`). PR #11307 targets the former. RDMACM-based wireup remains unaffected; fixing that path would require either a patch to `librdmacm` itself or an opt-out flag in UCX's connection manager.

4 Contrasting Production Designs

4.1 Apple MLX PR #3094 (JACCL distributed backend)

Apple's MLX framework added ring-topology distributed training support via a custom "JACCL" backend [3]. JACCL chose *CPU-mediated* collectives rather than async RDMA: sender and receiver are tightly coupled via sequential send/recv operations to prevent data corruption. Reported throughput is +124% on 4-node Devstral2 inference (135 → 303 tokens/sec on the prompt phase).

The tradeoff: the tight coupling eliminates entire classes of race conditions (helpful for a research framework where correctness trumps raw throughput), but it also serialises transfers and prevents overlap. This is the opposite pole of the async-CQ-with-priorities design UCX targets; libscuffedrdma's PMP controller is specifically what makes async overlap manageable when multiple priority classes share the same link.

4.2 Pure Storage TurboQuant + FlashBlade

Pure Storage's blog post [4] reports a production deployment of TurboQuant KV cache compression paired with FlashBlade object storage, achieving up to 10× faster KV cache restores (5–6× compression, sustained 900–1000 MB/s per GPU on DGX A100-40GB). The transport is RDMA plus GPUDirect Storage into GPU memory; the compression layer is TurboQuant.

This is relevant to the thesis for two reasons. First, it demonstrates that the TurboQuant-over-RDMA combination is already shipping in a production storage product, not a research prototype. Second, it clarifies where libscuffedrdma's contribution sits: FlashBlade moves compressed bytes fast and NIXL abstracts the transport, but neither classifies traffic by priority class or routes across a dual QP pool. The WFA/PMP layer is the part that remains libscuffedrdma's.

4.3 NVIDIA HPC-X

NVIDIA HPC-X [5] is the reference downstream distribution that bundles Open MPI, UCX, SHARP, UCC, and the NCCL UCX plug-in into a single optimised toolkit. HPC-X ships with ConnectX firmware and Quantum switch integrations; the SHARP component in particular is NVIDIA-fabric-only because it offloads collective operations onto the switch silicon. HPC-X is the correct choice for an all-NVIDIA datacenter cluster.

5 CI Status and Infrastructure Flakiness

At time of writing, CI status across the six PRs is dominated by infrastructure flakiness on the `swx-rain03` RoCE test node (UMR CQ poll timeouts, UD endpoint timeouts, DC performance-envelope timeouts). The same set of jobs fails across every PR regardless of the code change, confirming that the failures are independent of the patches under test. PR #11304 is currently in a closed state after a simultaneous force-push and close event on 2026-04-04; whether to reopen it is deferred to Section 8 because the user has not confirmed whether the close was intentional.

A separate survey of the ten most-recently-merged PRs in `openucx/ucx` shows all of them merged with 151/152 checks green, so the UCX maintainer workflow is to force-push-to-retrigger until clean rather than to override red RoCE checks. The current bottleneck is therefore scheduling (maintainer attention + infrastructure stability) rather than code correctness.

No comments have been posted on any of the six PRs during this update cycle: the author's preference is to keep the PR trackers focused on code review and document context on the thesis side.

6 Security Audit Findings

Triggered by a memory-safety bug the author discovered in libmesh-rdma's `mpi/mesh_mpi_pt2pt.c` (`malloc(rhdr->length) + memcpy` using a network-supplied length without validation against `wc.byte_len`), this section documents a parallel audit of both the six UCX PRs and the libscuffedrdma middleware.

6.1 UCX PRs — all clean

All six PRs were audited against the libmesh-rdma pattern and nine related classes (integer overflow, signed/unsigned mismatch, NULL dereference, buffer-over-read in address parsing, TCP bootstrap trust, traffic class validation, pack/unpack symmetry, exception handling, and `wc.byte_len` validation). **No findings.** Confidence is high across all six PRs. The changes are either configuration threshold logic (#11304), local state comparison (#11305), send-side application data only (#11306), test fixes (#11307), API additions (#11308), or bit-field isolation (#11309). None introduce new network-data-handling code paths.

One pre-existing concern, noted but *not* in scope for these PRs: `ucp_am_long_first_handler()` in `src/ucp/core/ucp_am.c` sums several network-supplied length fields without an overflow check. This is unrelated to the thesis PRs and would require a separate upstream patch.

6.2 Middleware — four high-severity findings

Four places in `/home/nathan/ScuffedRDMA/middleware/` read a network-supplied length and passed it to `recv()`, `frombuffer()`, or an array allocation without bounds checking. All four are the direct Python analogue of the libmesh-rdma bug that triggered the audit.

The attack model is trusted-peer on a management network, so these findings are defense-in-depth rather than active-exploitation targets. They matter for two reasons: a crashing handshake during a benchmark run is a thesis-result-ruining event even without an adversary, and the same code is on a path toward multi-tenant deployment (Configurations D and E from Update 4) where crafted peers become a real concern.

Finding	Fix	Severity
#1: `roce_transport.py` JSON handshake unbounded recv	Replaced entire JSON code path with fixed-size 64-byte binary `QpInfo` via `rdma_bootstrap` port	Critical
#2: `vllm_connector.py` unbounded `k_len + v_len` in KV layer transfer	Added `MAX_KV_LAYER_BYTES = 500MB` constant and pre-recv bounds check; wrapped `reshape()` in try/except on `ValueError`	Critical
#3: `sae_steering.py` unbounded `nnz` in `SparseVector.from_bytes`	Added `MAX_SPARSE_NNZ` / `MAX_SPARSE_DIM` bounds, explicit buffer length check before `frombuffer`; narrowed the `except` in `sync_from_remote` to only swallow `ValueError`	Critical
#4: `transport_tcp_sim.py` unbounded `resp_len` in async RDMA Read	Added `MAX_RDMA_READ_BYTES = 1GB` bound before `readexactly`	High

Table 1: Middleware findings and their remediation. All four are closed as of this update.

7 Middleware Hardening: Adapting libmesh-rdma

Alongside the audit, three new modules were added to `middleware/`, each adapting a production-tested piece of libmesh-rdma's C code into Python. Attribution is in a `LICENSE-THIRD-PARTY` file at the middleware root. The MIT license permits redistribution and adaptation with notice.

7.1 `rdma_gid_discovery.py`

Ports `mesh_rdma_find_ipv4_gid_index()` and helpers from `src/mesh_rdma_core.c`. Replaces the hardcoded `gid_index = 0` in `roce_transport.py` with a scan of the GID table that detects IPv4-mapped GIDs (10 zero bytes + 0xffff + 4 bytes of IPv4). When a `preferred_ip` is given, the highest matching index wins (RoCEv2 lives above RoCEv1 in the GID table). Raises `RuntimeError` with a diagnostic summary if no IPv4-mapped GID is found.

7.2 `rdma_bootstrap.py`

Ports the `mesh_rdma_qp_info_t` struct and `mesh_rdma_send_handshake` / `mesh_rdma_accept_handshake` from `src/mesh_rdma_core.c`. The wire format is 64 bytes of fixed-layout, network-byte-order struct: QPN, PSN, 16-byte GID, IPv4 address, GID index, MTU, and 28 bytes of padding for binary compatibility with the C struct. The Python API is two functions (`send_handshake`, `accept_handshake`) and one dataclass (`QpInfo`) with `pack()` / `unpack()` methods. A `_recv_exact` helper handles TCP's partial-read semantics, which the previous JSON-based code ignored.

This module is what closes Security Finding #1 from Table 1: because the wire format is fixed size, there is no length field for a crafted peer to manipulate, so the unbounded-`recv` class of bug is gone by construction.

7.3 `rdma_qp_state_machine.py`

Ports `mesh_rdma_create_qp` and `mesh_rdma_connect_qp`. Wraps pyverbs's `QP` with a `QueuePair` class that exposes `to_init() / to_rtr() / to_rts() / verify_rts() / close()`. Each transition is wrapped in a retry loop with exponential backoff (10, 20, 40, 80, 160 ms), and `verify_rts()`

queries the QP state after the final transition to catch silent failures. `close()` forces a transition back to RESET before `QP.close()` to avoid the `rdma_rxe` zombie state documented in Update 4 Section 9.5.

The key detail ported from libmesh-rdma is that `to_rtr()` sets the GRH fields (`dgid`, `sgid_index`, `hop_limit`) directly on the `AHAttr` rather than allocating an `AH` object. This is what makes the wireup work on direct-connect RoCE: the kernel never has to resolve an ARP entry because the destination GID is embedded in the QP state machine transition.

7.4 Integration with `roce_transport.py`

The existing `RoCETransport.connect()` method was retargeted to use the three new modules: `find_ipv4_gid_index` picks the GID index, `send_handshake` or `accept_handshake` exchanges `QpInfo` over TCP, and a `QueuePair` handles the state machine. The old JSON handshake, the hardcoded `gid_index = 0`, and the inline RTR/RTS transitions are all deleted. Net line change is approximately −80 LOC inline +30 LOC call sites +300 LOC across the three new modules, plus a test file.

7.5 Testing

`middleware/tests/test_rdma_bootstrap.py` contains 17 tests: 16 pure Python (wire format, handshake loopback, GID helpers, and regression tests for the three non-handshake security findings) and one hardware-dependent smoke test that verifies `QueuePair.to_init()` against real `rxe0`. The full RTR/RTS loopback test is guarded so that it skips on SoftRoCE builds where the kernel refuses loopback GRH transitions with EINVAL (a known rxe limitation that does not apply to real ConnectX hardware). On the local testbed, 16 tests pass and 1 skips.

8 Next Steps and Open Questions

Carried over from Update 4:

1. Multi-threaded concurrent workload on SoftRoCE to demonstrate head-of-line blocking elimination.
2. vLLM `KvConnector` integration for end-to-end prefill/decode over libscuffedrdma.
3. Port the data path from pyverbs to C++ or Rust to remove the GIL bottleneck.

New this update:

4. Confirm PR #11307 behavior on a real direct-connect RoCE ring once ConnectX hardware is available, to verify the RDMACM footnote in Section 3 holds.
5. Decide whether PR #11304 should be reopened; the close event was simultaneous with a force-push on 2026-04-04 and looks accidental, but this has not been confirmed.
6. Decide whether to responsibly disclose the `mesh_mpi_pt2pt.c` vulnerability upstream to `autoscriptlabs/lib` The project is MIT-licensed and in active development, so standard responsible-disclosure norms apply.
7. Run the full `QueuePair` loopback RTR test on real ConnectX hardware to confirm the SoftRoCE skip is a rxe quirk and not a wrapper bug.

References

[1] autoscriptlabs, "libmesh-rdma + libmesh-mpi: RDMA networking for consumer GPU clusters — no managed switch required." GitHub, 2026. https://github.com/autoscriptlabs/libmesh-rdma

[2] Ok-Pomegranate1314, "Custom MPI over RDMA for direct-connect RoCE — no managed switch, no UCX, no UD. 55 functions, 75 KB." r/HPC, 2026. https://www.reddit.com/r/HPC/comments/1ruzjw3/custom_mpi_over_rdma_for_directconnect_roce_no/

[3] ml-explore, "MLX PR #3094: JACCL distributed backend." GitHub, 2026. https://github.com/ml-explore/mlx/pull/3094

[4] R. Alvarez, "Up to 10× faster KV cache restore: TurboQuant meets FlashBlade." Pure Storage Blog, 2026. https://blog.purestorage.com/purely-technical/up-to-10x-faster-kv-cache-restore-turboquant-meets-flashblade/

[5] NVIDIA, "HPC-X: High-Performance Computing Software Toolkit." NVIDIA Networking, 2026. https://developer.nvidia.com/networking/hpc-x

[6] Introl, "GPU virtualization: maximizing utilization in multi-tenant environments." Introl Blog, 2025. https://introl.com/blog/gpu-virtualization-maximizing-utilization-multi-tenant-environments